AULÃO BV · AULA TÉCNICA · 2 HORAS

A Memória do Agente

Implementando RAG na Agent Platform do Google — do notebook à nuvem

O FIO DA AULA · "A GRANDE MUDANÇA"

Nosso agente negociador já tem memória — mas ela mora numa garagem: um ChromaDB no notebook. Hoje é o dia da mudança: vamos levar essa biblioteca inteira para a sede — a **Agent Platform do Google** — sem perder uma ficha sequer no caminho.

VOCÊ SAI DAQUI COM

- Um índice criado, implantado e consultável no Vector Search
- O agente do projeto conectado à nuvem
- O critério para escolher RAG Engine × Vector Search

Projeto de referência: agente-3-the-memory · Público técnico · Storytelling: Explicando em detalhes

ABERTURA · 00:00–00:10

O agente que esquecia tudo

A CENA

Um cliente negocia financiamento com nosso agente. Recusa a taxa três vezes, explica que só fecha abaixo de 1,0%… e volta na semana seguinte.

O agente o cumprimenta como um estranho. Oferece a mesma taxa. Repete os mesmos erros.

Um LLM é um gênio com amnésia: brilhante na conversa, incapaz de lembrar da anterior. A janela de contexto acaba, e tudo evapora.

Memória não é um truque de prompt. É infraestrutura.

ONDE PARAMOS (o projeto)

agente-3-the-memory

Agente negociador com:

- Memória curta: sessão + checkpoint (FSM de negociação)
- Memória longa: RAG — insights e políticas em vector store
- Pipeline: loaders → chunking → embedding → vector store

Hoje: backend chroma (local). Ao final da aula: backend vertex (nuvem).

Pergunta para a sala: "o que a SUA aplicação esquece hoje que custa dinheiro?"

AGENDA

O mapa das 2 horas

00:00	**Abertura**	O agente que esquecia tudo — por que memória é infraestrutura
00:10	**Ato 1 · Dominando RAG**	Picotar, endereçar, guardar, buscar — os 4 desenhos
00:25	**Ato 2 · O mapa da plataforma**	Agent Platform por dentro: RAG Engine × Vector Search
00:40	**Ato 3 · Mão na massa I**	Dados no GCS e o formulário “Criar índice”, campo a campo
01:05	**Respiro + Q&A**	Enquanto o índice “assa” — perguntas de bolso
01:10	**Ato 4 · Mão na massa II**	Endpoint, deploy, find_neighbors — e o agente ligado na nuvem
01:30	**Ato 5 · A estrada gerenciada**	RAG Engine, Memory Bank e Sessions — quando deixar o Google dirigir
01:45	**Ato 6 · Produção sem susto**	Custos, avaliação, pegadinhas e limpeza
01:57	**Fechamento**	A cadeia completa da memória — e o compromisso de segunda-feira

1

ATO 1 · 00:10–00:25

Dominando RAG

Quatro desenhos explicam tudo: picotar o livro, dar um endereço a cada ficha, guardar na estante certa e pedir ao bibliotecário os vizinhos mais parecidos.

Uma biblioteca em quatro desenhos

1 · PICOTAR

Chunking

Ninguém acha nada num livro de 500 páginas. Recortamos em fichas de leitura — pedaços pequenos, com uma sobreposição para não cortar frase no meio.

No projeto: 512 caracteres, overlap 64, estratégia recursive.

2 · ENDEREÇAR

Embedding

Cada ficha ganha um endereço no “mapa das ideias”: uma lista de números onde assuntos parecidos moram perto. “Taxa de juros” fica vizinha de “custo do empréstimo”.

Modelos Vertex: gemini-embedding-001 (até 3.072 dims).

3 · GUARDAR

Índice (Vector Search)

As fichas vão para estantes organizadas por vizinhança, não por ordem alfabética. É isso que torna a busca instantânea mesmo com bilhões de fichas.

Algoritmo Tree-AH: busca aproximada, rápida e barata.

4 · BUSCAR

Retrieval + Geração

A pergunta também vira um endereço. O bibliotecário traz as k fichas mais próximas — e o LLM responde usando SÓ o que está nelas, citando a fonte.

find_neighbors(query, k) → contexto → prompt → Gemini.

Indexar = arrumar a biblioteca (uma vez, em lote). Consultar = perguntar ao bibliotecário (a cada mensagem).

Embeddings: o GPS do significado

EXPICANDO DETALHADAMENTE

Imagine um mapa gigante onde cada frase é um pingo. Frases que falam da mesma coisa caem pertinho; frases diferentes caem longe.

O embedding é a latitude/longitude desse pingo — só que com centenas de coordenadas em vez de duas.

“Qual a distância entre dois pingos?” é a única conta que o Vector Search faz — bilhões de vezes por segundo.

PARA O TIME TÉCNICO

Dimensão é contrato: o nº de dims do índice TEM de ser igual ao do modelo de embedding. 768 no projeto (text-multilingual-embedding-002); gemini-embedding-001 vai de 128 a 3.072 (Matryoshka).
Task type importa: documento é indexado como RETRIEVAL_DOCUMENT; a pergunta, como RETRIEVAL_QUERY. O modelo otimiza cada lado — e a busca melhora de graça.
Trocou de modelo? Reindexe tudo. Mapas de modelos diferentes não são compatíveis — nem se a dimensão coincidir.

No código do projeto: embed_texts(texts, for_query=True|False) já alterna o task type automaticamente.

Chunking: como picotar sem perder o sentido

fixed

corta a cada N caracteres, com overlap. Simples e previsível — mas corta frases ao meio.

by_paragraph / by_sentence

respeita as fronteiras naturais do texto. Fichas mais “inteiras”, tamanhos irregulares.

recursive ✓ (a do projeto)

tenta parágrafo → linha → sentença → palavra. O melhor dos dois mundos: fichas coerentes e limitadas.

by_tokens

conta tokens (tiktoken), não caracteres — alinha o chunk ao custo real do modelo.

`config/indexing.yaml`

```
chunking:
  chunk_size_chars: 512
  overlap_chars: 64
  strategy: recursive

embedding:
  backend: vertex
  model: text-multilingual-
         embedding-002
  batch_size: 5
```

Regra de bolso: chunk pequeno acha melhor, chunk grande explica melhor. 512/64 é o equilíbrio do nosso caso bancário.

2

ATO 2 · 00:25–00:40

O mapa da plataforma

A Agent Platform é a “cidade dos agentes” do Google Cloud. Antes de construir, um tour pelos bairros — e a decisão mais importante da aula: qual estrada de RAG seguir.

ATO 2 · O TOUR

Agent Platform: o que é cada coisa no menu

CRIAR

Agent Garden — vitrine de agentes prontos e exemplos para clonar
ADK — o framework de código para construir agentes (Python)
Servidores MCP — ferramentas externas plugáveis via protocolo MCP
Mecanismo RAG — RAG gerenciado: corpus, import e retrieval prontos
Vector Search — o banco vetorial: índices + endpoints (nossa aula)

ESCALA

Implantações — onde os agentes rodam (Agent Engine runtime)
Memory Bank — memória de longo prazo gerenciada por usuário
Sessões — memória de curto prazo: estado e histórico da conversa

GOVERNANÇA

Agent Registry — inventário de agentes da organização
Policies / Security — o que cada agente pode e não pode fazer
Gateways / Topology — portas de entrada e mapa de dependências

Memória curta = Sessões. Memória longa = Memory Bank ou o SEU RAG. Hoje construímos a versão "o seu RAG".

ATO 2 · A DECISÃO

Duas estradas para o mesmo RAG

MECANISMO RAG (RAG Engine)

“Comprar o bolo pronto”

Você cria um corpus, importa arquivos (GCS/Drive) e pronto: chunking, embedding e indexação acontecem por conta da plataforma.

- Ideal quando: quer velocidade, não precisa customizar o pipeline, documentos “bem-comportados”.
- Você abre mão de: controle fino de chunking, metadados e pós-processamento.

VECTOR SEARCH ✓ NOSSA ESTRADA

“Assar o próprio bolo”

Você controla cada etapa: chunking próprio, embeddings com task type, JSONL no GCS, índice, endpoint, consulta.

- Ideal quando: já existe pipeline (nosso caso!), precisa de metadados ricos, filtros, min_similarity, paridade com o ambiente local.
- Você assume: mais peças para operar — e é por isso que a aula percorre TODAS.

Dá para ter os dois: o RAG Engine aceita o Vector Search como banco vetorial por trás do corpus (Ato 5).

Do notebook à nuvem: o que muda no projeto

1 · PIPELINE LOCAL

`python -m src.indexing loaders → chunking → embedding (vertex)`

→

2 · GCS

`JSONL {id, embedding} gs://bucket/ vertex_batch/`

→

3 · ÍNDICE

`Vector Search Index Tree-AH · 768 dims batch update`

→

4 · ENDPOINT

`Index Endpoint público + deploy do índice (deployed_index_id)`

→

5 · AGENTE

`memory_gateway find_neighbors(k=3) → prompt do Gemini`

O QUE NÃO MUDA (a beleza da abstração)

O agente não sabe onde a memória mora. Ele chama `memory_gw.search_customer_insights(query)` — e o gateway decide o backend: chroma | vertex | mock, via `config/memory_policy.yaml` **. A mudança inteira é 1 linha de config + variáveis de ambiente.**

3

ATO 3 · 00:40–01:05

Tutorial: do Storage ao índice

Construção guiada, junto com a turma: criar o depósito (Cloud Storage), encaixotar as fichas (JSONL) e preencher o formulário “Criar índice” — com um mergulho de verdade em CADA campo. Sem decoreba: cada escolha tem um porquê.

ATO 3 · PASSO 0

Antes de tudo: o alvará da obra

Projeto + billing	Vector Search é serviço pago. Sem billing habilitado, nada acontece — e com billing, TUDO que ficar de pé cobra por hora (limpeza no Ato 6).
APIs habilitadas	aiplatform.googleapis.com, storage.googleapis.com, compute.googleapis.com. O botão “Ativar APIs” do console faz o mesmo que o gcloud.
IAM	Quem cria: Vertex AI User + Storage Admin no bucket. Em produção: uma service account dedicada, nunca a sua conta pessoal.
Uma região para tudo	Bucket, índice e endpoint na MESMA região (ex.: us-east1). Misturar regiões = erro na criação ou latência e custo extras.

```
gcloud services enable aiplatform.googleapis.com storage.googleapis.com compute.googleapis.com
```

ATO 3 · PASSO 1

Cloud Storage: o depósito da mudança

NO CONSOLE (com a turma)

1. Menu ☰ → Cloud Storage → Buckets → Criar.
2. Nome: único no mundo inteiro (ex.: meu-projeto-rag-bv). Minúsculas, sem espaço.
3. **Tipo de local: Região → us-east1 — a MESMA que o índice usará. Essa é a regra que não se negocia.**
4. Classe Standard, acesso uniforme, sem acesso público. Criar.
5. Dentro do bucket, crie a pasta vertex_batch/ — é ELA que o índice vai ler (contentsDeltaUri).

PELO TERMINAL (mesma coisa)

```
gsutil mb -l us-east1 \
  -p $PROJECT_ID \
  gs://$PROJECT_ID-rag-bv

gsutil cp vectors.jsonl \
  gs://$PROJECT_ID-rag-bv/\
  vertex_batch/
```

Analogia: o bucket é o depósito na porta da biblioteca nova. O caminhão (índice) só aceita caixas que já estejam NO depósito — nunca direto da sua mão (upload local não existe na criação do índice).

ATO 3 · PASSO 2

Encaixotar a biblioteca: o JSONL de vetores

O FORMATO (uma ficha por linha)

```
{"id": "chunk-001",
 "embedding": [0.12, -0.08, …]}
{"id": "chunk-002",
 "embedding": [0.55, 0.31, …]}
```

- id único por chunk; embedding com EXATAMENTE as dimensões do índice.
- O índice não guarda o texto — só id + vetor. O texto fica no nosso content store (id → conteúdo), consultado após a busca.
- A pasta do GCS com esses arquivos é o contentsDeltaUri do índice.

NO PROJETO, É UM COMANDO

```
# .env: GOOGLE_CLOUD_PROJECT,
#  GOOGLE_CLOUD_LOCATION,
#  VECTOR_SEARCH_GCS_BUCKET…

python -m src.indexing \
  --config config/indexing.yaml \
  --input data/lab \
  --output out/chunks.json \
  --push
```

Com backend: vertex, o --push gera o JSONL, sobe para o bucket e dispara o update do índice — o vector_store.py faz a mudança inteira.

ATO 3 · PASSO 3

O formulário “Criar índice”: o mapa geral

Nome / Descrição

só para humanos acharem depois. Padrão do time: rag-<projeto>-<versão>.

Região

us-east1 no nosso caso — a MESMA do bucket. Grave essa regra.

URI da pasta do GCS

a pasta (não o arquivo!) onde estão os JSONL: gs://bucket/vertex_batch/.

Algoritmo: Tree-AH

árvore + hashing assimétrico: busca aproximada. Troca 0,1% de precisão por 100x de velocidade.

Dimensões

768 aqui (text-multilingual-embedding-002). Errou esse número? A criação falha — ou pior, a busca vira ruído.

Vizinhos aproximados

150: quantos candidatos o índice separa antes do reordenamento fino. Regra: ~10x o k que você pedirá.

Método: Lote (batch)

atualiza por remessas via GCS — nosso caso. Streaming = upsert em segundos (e é o que o RAG Engine exige).

Tamanho do fragmento

Médio (20 GiB): shards são as “caixas” em que o índice é dividido; definem a máquina do deploy.

Este é o mapa. Os próximos 6 slides mergulham em cada campo — preencham junto comigo no console.

ATO 3 · CAMPO: TIPO DE ALGORITMO

Tree-AH ou Força bruta?

TREE-AH ✓ (produção)

Busca APROXIMADA (ANN), baseada no ScaNN do Google: uma árvore agrupa vetores parecidos em “gavetas” (nós-folha) e o hashing assimétrico compara rápido dentro delas.

Não olha todos os vetores — olha só as gavetas promissoras. Resultado: milissegundos mesmo com bilhões de itens, errando raríssimo (recall típico > 95%).

Use quando: produção, base grande, latência importa. Ou seja: quase sempre.

FORÇA BRUTA (gabarito)

Busca EXATA: compara a consulta com TODOS os vetores, um a um. Sem parâmetros para configurar — e sem atalhos.

Preciso por definição, lento por definição. Custo cresce linear com a base: dobrou a base, dobrou o tempo.

Use quando: base pequena, ou como GABARITO — você roda força bruta numa amostra para medir o recall do Tree-AH. Nunca em produção com latência apertada.

ANALOGIA · Procurar um amigo num estádio: força bruta olha rosto por rosto nas 80 mil cadeiras (acha com certeza, demora horas); Tree-AH pergunta ao guichê em qual setor ele se sentou e revista só aquele setor (acha em minutos — e ele quase nunca está fora do setor do ingresso).

Os dois números que o formulário exige

DIMENSÕES *

Quantas coordenadas tem cada vetor. Não é escolha sua — é do MODELO de embedding:

- text-multilingual-embedding-002 ⟶ 768 (a do projeto)
- gemini-embedding-001 ⟶ 3.072 por padrão (aceita 128–3.072, estilo Matryoshka)
- MiniLM local do lab ⟶ 384

Dimensão é contrato: índice 768 + JSONL 3.072 = criação falha. E mais dimensão = mais nuance semântica, porém mais memória, custo e latência.

CONTAGEM ESTIMADA DE VIZINHOS *

A busca do Tree-AH tem 2 tempos: (1) a fase aproximada separa N candidatos; (2) a reordenação EXATA reordena só esses N e entrega o top-k.

Este campo é o N: quantos candidatos separar antes do reordenamento. Default 150 — bom equilíbrio precisão × latência.

Regra de bolso: ~10× o k que sua aplicação vai pedir (pedimos k=3; 150 sobra com folga).

Analogia: o bibliotecário separa 150 fichas "provavelmente boas" na bandeja e aí, com calma, escolhe as 3 melhores.

ATO 3 · CAMPO: MÉTODO DE ATUALIZAÇÃO

Lote ou Streaming: o ritmo da sua verdade

LOTE (BATCH) ✓ nossa aula

Atualiza por remessas: novo JSONL no GCS ⟶ operação de update ⟶ o índice reconstrói (minutos a horas).

Use quando a verdade muda em ciclos: política de crédito semanal, tabela de tarifas mensal, base de contratos consolidada à noite.

Mais barato e mais simples de operar. É o ritmo certo para documentos corporativos.

STREAMING

Upsert registro a registro: o dado novo fica consultável em segundos, sem reconstruir o índice.

Use quando a verdade muda o tempo todo: estoque, preços dinâmicos, feed de notícias, memória viva de conversas.

Atenção: é o método que o Mecanismo RAG exige (STREAM_UPDATE) — e custa mais por essa prontidão.

ANALOGIA · Lote é o caminhão de reposição que chega toda segunda de manhã. Streaming é o repositor que coloca cada produto na prateleira assim que ele chega na doca. A pergunta de arquitetura: o seu cliente pode esperar até segunda?

Fragmentos (shards): em quantas caixas dividir

O índice é dividido em partes iguais — os fragmentos (shards). No serving, **cada fragmento vira um nó (máquina) separado**: o tamanho escolhido define quantas máquinas, de que porte — e a fatura.

PEQUENO

2 GiB por fragmento

Bases pequenas (até poucos milhões de vetores). Máquinas menores, custo mínimo — o tamanho certo para POCs, labs e o nosso caso.

MÉDIO

20 GiB por fragmento

O meio-termo (default do console). Dezenas de milhões de vetores por fragmento em máquinas intermediárias.

GRANDE

50 GiB por fragmento

Escala bilhões-de-vetores. Menos fragmentos para a mesma base, máquinas parrudas por nó — para quem tem tráfego e orçamento à altura.

CUIDADO · o tamanho do fragmento é definido na CRIAÇÃO e não muda depois — mudou de ideia, recria o índice. Dimensione pelo tamanho que a base TERÁ, não pelo que ela tem hoje.

ATO 3 · AVANÇADO: MEDIDA DE DISTÂNCIA

As quatro réguas de “parecido”

PRODUTO ESCALAR ✓ default

Multiplica coordenada por coordenada e soma. Mede alinhamento de direção E magnitude. Com vetores normalizados, dá o MESMO ranking do cosseno — com a conta mais barata que existe. É a escolha para embeddings de texto.

DISTÂNCIA DO COSSENO

Só o ângulo entre os vetores, ignorando o comprimento. Conceitualmente ótima para texto — mas o console avisa: “altamente não recomendado”, porque produto escalar + normalização entrega o mesmo resultado gastando menos. Normalize e use dot.

EUCLIDIANA (L2)

A régua da geometria: a linha reta entre dois pontos (ao quadrado, na prática). Faz sentido quando a POSIÇÃO absoluta no espaço importa — visão computacional, dados de sensores, features numéricas cruas.

MANHATTAN (L1)

A distância do taxista: soma dos deslocamentos eixo a eixo, como quarteirões de uma cidade. Mais robusta a valores extremos (outliers) em dimensões isoladas; nicho — dados esparsos ou features com escalas distintas.

Regra: a distância deve casar com o que o MODELO de embedding otimizou. Texto (nossos modelos Vertex): produto escalar com vetores normalizados. Ponto final.

ATO 3 · AVANÇADO: OS ÚLTIMOS TRÊS CAMPOS

Normalização e os botões do Tree-AH — e Criar

TIPO DE NORMALIZAÇÃO

Nenhum (default) ou UNIT_L2_NORM — encolhe/estica cada vetor para comprimento 1, sobrando só a direção.

É o par perfeito do produto escalar: normalizou, dot ≡ cosseno.

Nosso pipeline já normaliza no embedding → aqui pode ficar "Nenhum". Se não tiver certeza, UNIT_L2_NORM é o cinto de segurança.

EMBEDDINGS POR NÓ-FOLHA

1.000 (default): quantas fichas cabem em cada "gaveta" da árvore.

Gavetas maiores = menos gavetas para escolher (árvore mais rasa), porém mais fichas para comparar dentro de cada uma.

1.000 serve para a imensa maioria; só mexa com milhões de vetores E medição de latência na mão.

FRAÇÃO DE FOLHAS PESQUISADAS

0,05 (default): a cada busca, abrimos 5% das gavetas — as mais promissoras segundo a árvore.

É O botão recall × latência: 0,05 → 0,10 acha mais e demora mais; 0,05 → 0,01 voa e erra mais.

Ajuste DEPOIS, medindo hit rate contra o gabarito força bruta (Ato 6) — nunca no chute.

Agora sim: CRIAR. Status "Criando…" → "Pronto" em minutos a ~1h. Deixem criando — o respiro é agora, e o índice de ontem nos espera.

RESPIRO + Q&A · 01:05–01:10

Enquanto o índice assa…

TEMPOS REAIS (planeje a aula e a vida)

- Criar índice (batch): minutos a ~1 hora, conforme o volume.
- Primeiro deploy no endpoint: até ~30 minutos. Os seguintes: bem mais rápidos.
- Update batch de um índice vivo: minutos a horas — por isso existe streaming.
- **Na aula: usamos um índice já "Pronto", criado ontem — como um chef de TV que tira o bolo assado do forno de baixo.**

PERGUNTAS DE BOLSO

"Que documento da sua área você indexaria primeiro — e quem pode vê-lo?"

"Seu caso é batch (política de crédito muda por semana) ou streaming (estoque muda por segundo)?"

4

ATO 4 · 01:10–01:30

Mão na massa II: servir a memória

Estantes prontas não atendem ninguém. Falta o balcão (endpoint), contratar o bibliotecário (deploy) — e ensinar o agente a fazer a pergunta certa (find_neighbors).

Endpoint e deploy: o balcão e o bibliotecário

INDEX ENDPOINT — o balcão

O índice NÃO recebe requisições. Quem atende é o endpoint — um serviço com URL, autenticação e máquinas por trás.

- Público (com IAM) para começar; VPC/Private Service Connect quando a rede do banco mandar.
- Console: aba “Endpoints de índice” → Criar. Mesma região do índice.

```
gcloud ai index-endpoints create
  --display-name=rag-endpoint
  --public-endpoint-enabled
```

DEPLOY — a contratação

Implantar o índice no endpoint é dar as chaves da estante ao bibliotecário. Só depois disso dá para consultar.

- Você escolhe o deployed_index_id (ex.: rag_bv_deployed) — é ELE que vai nas consultas. Um endpoint pode servir vários índices.
- É no deploy que nasce a máquina (e o custo por hora). Botão “Implantar” na lista de índices — o mesmo dos prints.

A tríade das consultas: ENDPOINT_ID + DEPLOYED_INDEX_ID + região. Anote os três.

ATO 4 · PASSO 6

find_neighbors: a pergunta ao bibliotecário

```
from google.cloud import aiplatform

aiplatform.init(project=PROJECT, location="us-east1")
endpoint = aiplatform.MatchingEngineIndexEndpoint(ENDPOINT_ID)

query_vec = embed_texts([pergunta], for_query=True)[0]

resp = endpoint.find_neighbors(
    deployed_index_id="rag_bv_deployed",
    queries=[query_vec],
    num_neighbors=3,
)
for n in resp[0]:
    print(n.id, n.distance)    # id da ficha + proximidade
```

LEIA COM ATENÇÃO

- A pergunta vira vetor com for_query=True (RETRIEVAL_QUERY) — mesmo modelo da indexação.
- Volta id + distance — NÃO volta o texto. O texto sai do nosso content store (id → conteúdo).
- similaridade = max(0, 1 − distance); descartamos o que ficar abaixo de min_similarity_score.
- k=3 vizinhos: o suficiente para contexto sem inundar o prompt.

Ligando o agente na memória da nuvem

A MUDANÇA INTEIRA EM CONFIG

```
# config/memory_policy.yaml
vector_search:
  backend: vertex      # era: chroma
  max_documents: 3
  min_similarity_score: 0.6

# .env
GOOGLE_CLOUD_PROJECT=meu-projeto
GOOGLE_CLOUD_REGION=us-east1
VECTOR_SEARCH_ENDPOINT_ID=5757…
VECTOR_SEARCH_INDEX_ID=8648…
VECTOR_SEARCH_DEPLOYED_INDEX_ID=rag_bv_deployed
VECTOR_SEARCH_GCS_BUCKET=meu-bucket
```

ENGENHARIA DE GENTE GRANDE (já no projeto)

- Retry com backoff (tenacity): 3 tentativas, espera 2–10s — rede falha, a memória não pode derrubar a conversa.
- Degradação graciosa: se a busca falhar de vez, retorna “” e o agente segue sem RAG — pior resposta, nunca resposta nenhuma.
- asyncio.to_thread: a busca roda fora do event loop — o agente não congela esperando a estante.
- **Teste de fumaça: python scripts/rag_query.py "tarifas da conta premium"**

5

ATO 5 · 01:30–01:45

A estrada gerenciada

Agora que você sabe assar o bolo, pode julgar o bolo pronto. O Mecanismo RAG (RAG Engine), o Memory Bank e as Sessões — o que a plataforma faz por você e o que ela cobra em troca.

ATO 5 · MECANISMO RAG

RAG Engine: corpus, import, retrieve, generate

1 · CORPUS

a “biblioteca gerenciada”: você cria e aponta o banco vetorial (ou usa o RagManagedDb default)

2 · IMPORT

ragFiles.import: arquivos do GCS ou Drive entram; chunking e embedding automáticos

3 · RETRIEVE

RetrieveContexts: a busca semântica pronta, sem gerenciar endpoint

4 · GENERATE

Gemini com o corpus como ferramenta: recupera e responde com base no contexto

O DETALHE QUE CONECTA AS DUAS ESTRADAS

O corpus aceita o SEU Vector Search como banco por trás — mas exige índice com STREAM_UPDATE e distância dot product ou cosseno. **Nosso índice batch do Ato 3 não serve para o RAG Engine — e está tudo bem: são estradas para casos diferentes.** Outros bancos aceitos: RagManagedDb (default), Vector Search 2.0, Feature Store, Pinecone, Weaviate.

Memory Bank e Sessões: o cérebro completo

SESSÕES — memória de curto prazo

O caderninho da conversa atual: histórico, estado, checkpoint. No projeto, é o Session Gateway (FSM: funnel_stage, proposed_rate, rejection_count).

Na plataforma: serviço gerenciado de Sessions — o mesmo conceito, sem manter banco próprio.

MEMORY BANK — memória de longo prazo gerenciada

O diário por usuário: a plataforma extrai fatos das conversas (“só fecha abaixo de 1,0%”) e os devolve nas próximas sessões.

É o nosso memory_gateway — versão gerenciada. O que construímos à mão hoje existe como serviço: e agora você sabe EXATAMENTE o que ele faz por dentro.

Curto prazo = a conversa. Longo prazo = o cliente. RAG = o conhecimento. Um agente sério precisa dos três.

ATO 6 · 01:45–01:57

Produção sem susto: custos e pegadinhas

O taxímetro

endpoint com índice implantado cobra POR HORA, com ou sem consultas. O esquecimento clássico: a demo de sexta virando fatura de segunda.

Limpeza (a ordem importa)

1) undeploy do índice → 2) delete endpoint → 3) delete índice → 4) esvaziar bucket. Undeploy primeiro; o resto falha se houver deploy ativo.

Atualização batch

novo JSONL no GCS + update com contentsDeltaUri. isCompleteOverwrite: false acrescenta; true substitui TUDO — leia duas vezes antes de usar.

⚠️ Dimensão errada

índice 768 × embedding 3072 = falha (ou lixo silencioso). Dimensão é contrato.

⚠️ Região trocada

bucket em us-central1, índice em us-east1: não cria. Uma região para tudo.

⚠️ Esqueceu o deploy

índice “Pronto” não é índice consultável. Sem deploy, find_neighbors não existe.

⚠️ Modelo trocado no meio

indexou com um modelo, consulta com outro: distâncias sem sentido. Reindexe.

ATO 6 · MEDIR ANTES DE ACREDITAR

RAG sem avaliação é fé, não engenharia

O GOLDEN SET

Um gabarito: perguntas reais + o chunk que DEVERIA voltar. No projeto: data/lab/golden_set.json.

Hit rate@k: em quantas perguntas o chunk certo apareceu no top-k? É a nota da biblioteca — simples de calcular, brutal de honesta.

python scripts/eval_retrieval.py roda o gabarito contra o backend configurado — local OU nuvem, mesma régua.

OS BOTÕES DE AJUSTE (nesta ordem)

- 1. Chunking (tamanho/estratégia) — o maior impacto quase sempre mora aqui.
- 2. min_similarity_score — sobe para cortar ruído, desce se faltar contexto (0.6–0.8 com embeddings Vertex).
- 3. k (max_documents) — mais vizinhos ≠ mais qualidade; prompt inflado confunde o LLM.
- 4. Fração de folhas (0.05 → 0.1) — só depois de esgotar os anteriores.

FECHAMENTO · 01:57–02:00

A mudança está completa



A FRASE PARA LEVAR

"Um LLM sem RAG responde de memória. Um agente com RAG responde com a biblioteca aberta na mesa — e agora a sua biblioteca mora num prédio que não pega fogo, não perde ficha e atende bilhões de consultas."

O compromisso de segunda-feira: escolha UM documento da sua área, rode o pipeline com backend vertex, crie o índice na SUA conta — e apague tudo no fim do dia (o taxímetro agradece).

ANEXO

Referências e material de apoio

Projeto da turma	agente-3-the-memory · branch rag-turma-2 · docs/tutorial_vertex_vector_search.md (passo a passo completo com gcloud e Python)
Vector Search	Documentação Google Cloud: visão geral, criar e gerenciar índices, implantar endpoints públicos, formato dos dados (JSONL), atualizar índice ativo
RAG Engine (Mecanismo RAG)	Visão geral do RAG Engine na Agent Platform; uso com Vector Search (exige STREAM_UPDATE); bancos suportados: RagManagedDb, Vector Search 2.0, Feature Store, Pinecone, Weaviate
Embeddings	gemini-embedding-001 (128–3.072 dims, task types RETRIEVAL_DOCUMENT / RETRIEVAL_QUERY); text-multilingual-embedding-002 (768 dims — a do projeto)
Custos	Preços do Vertex AI / Vector Search — índice implantado cobra por hora; sempre fazer undeploy + delete após laboratórios